

Web Caching and Conditional GET

Introduction to Web Caching

Web Caching (also called a **Proxy Server**) is a network entity that stores copies of recently requested web objects. When a browser requests content, instead of directly contacting the origin server, the request may be handled by the cache.

Working Principle

- Step 1:** The browser establishes a TCP connection to the **Web Cache** and sends all HTTP requests to it.
- Step 2:** If the requested object exists in the cache, it is immediately returned to the browser.
- Step 3:** If not, the cache requests the object from the **Origin Server**, stores a local copy, and returns it to the client.
- Step 4:** The cache acts as both **server** (to the client) and **client** (to the origin server).

Note: The origin server can specify caching policies via HTTP headers, e.g.,

- Cache-Control: max-age=<seconds>
- Cache-Control: no-cache

Note: A cache acts as both a **server** (to clients) and a **client** (to origin servers).

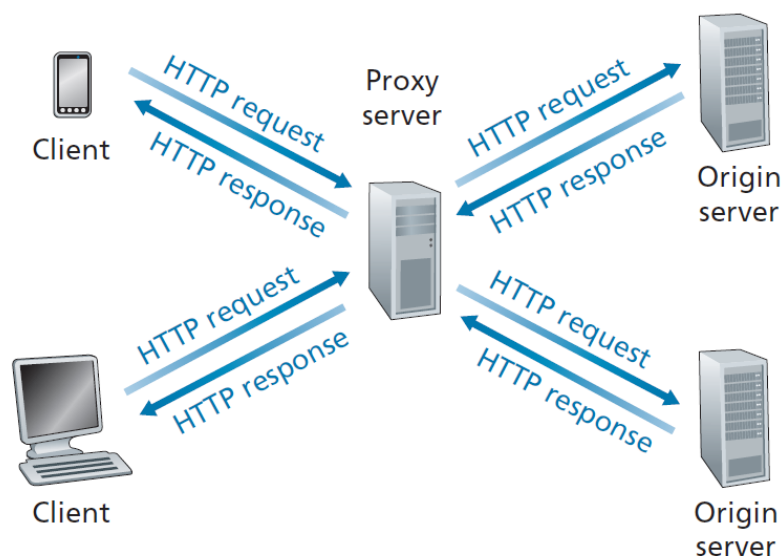


Figure: Client requests objects via a Web Cache (Proxy Server)

Benefits of Web Caching

- **Reduced Response Time:** Cache is closer to the client, speeding up requests.
- **Reduced Bandwidth Usage:** Reduces traffic on institutional access links.
- **Cost Efficiency:** Cheaper than upgrading high-speed links.
- **Improved Internet Performance:** Dense cache deployment improves global traffic handling.

Performance Scenario

Scenario Parameters

- Access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- Web object size: 100 Kbits
- Average request rate: 15 requests/sec
- Average data rate to browsers: 1.50 Mbps

Performance Without Caching

$$\text{Access link utilization} = \frac{\text{Data rate}}{\text{Link rate}} = \frac{1.50}{1.54} \approx 0.97$$

$$\text{LAN utilization} = \frac{100\text{Kbits} \times 15}{100\text{Mbps}} \approx 0.0015$$

End-to-end delay:

Delay = Internet delay + Access link delay + LAN delay $\approx$ 2 sec + minutes + microseconds

Observation: Extremely high access link delay makes response times unacceptable.

Option 1: Upgrading Access Link

- Upgrade link to higher bandwidth (expensive).
- Reduces utilization and delay:

$$\text{Access link utilization} = \frac{1.50}{\text{New rate}}$$

- End-to-end delay $\approx$ 2 sec (Internet delay dominates).

Option 2: Install a Web Cache

Assume **hit rate** = 0.4:

- 40% of requests served by cache with negligible delay (msec).
- 60% of requests satisfied by origin servers:

$$\text{Rate over access link} = 0.6 \times 1.50 \text{ Mbps} = 0.9 \text{ Mbps}$$

- Access link utilization:

$$\rho = \frac{0.9}{1.54} \approx 0.58$$

- Low queueing delay, average end-to-end delay:

$$\text{Avg. delay} = 0.4 \cdot (\text{cache delay}) + 0.6 \cdot (\text{origin delay}) \approx 0.4 \cdot 0.01 + 0.6 \cdot 2.01 \approx 1.2 \text{ sec}$$

Conclusion: Installing a web cache reduces average delay significantly at much lower cost than upgrading the access link.

Content Distribution Networks (CDNs)

- Deploy geographically distributed caches.
- Popular CDNs: **Akamai, Limelight, Google, Netflix.**
- Localizes content delivery, reduces load, improves performance.

Conditional GET

Caching introduces **stale objects** problem. Conditional GET ensures freshness.

Mechanism

A conditional GET request:

1. Uses **GET** method.
2. Contains **If-Modified-Since** header.

Example

- 1) Cache requests **kiwi.gif**.
- 2) Server responds with object + **Last-Modified: 9 Sep 2015**.
- 3) Later, cache sends:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-Modified-Since: Wed, 9 Sep 2015 09:23:24
```

4) If unchanged, server replies:

HTTP/1.1 304 Not Modified

avoiding unnecessary retransmission.

Summary

- Web caching reduces latency and bandwidth usage.
- Performance depends on hit rate; even moderate rates improve delay significantly.
- Installing a cache is cheaper than upgrading links.
- CDNs amplify caching benefits globally.
- Conditional GET maintains cache freshness.